

0.29.2. Revenir au départ avec `va_a` — doublé, deux positions

L'aller-retour du A (en haut), puis celui du B (en bas).

1. Ce qu'il faut comprendre

C'est le 0.29 et le 0.29.1 réunis : les deux allers-retours.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <va_a>` → [0.27.3. La fonction `va\_a\(\)` — aller jusqu'à une case](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1187 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      69 o ← la console, à l'allumage – la mise en route de la console
va_a()          176 o ← main() – une fonction fournie par le langage
main()          120 o ← démarrage – ta fonction
VBlank          8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage   28 o ← image() – attend l'image suivante
AttendreVBlank  16 o ← démarrage – attend que l'écran soit libre
AttendreAcces   12 o ← EffacerCases – attend de pouvoir écrire une case
AdresseCase     10 o ← poser(), effacer() – trouve l'adresse d'une case de l'écran
EffacerCases    12 o ← effacer() – efface des cases
CopierTuiles    18 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o ← démarrage – vide les deux cartes de l'écran
Tuiles          704 o ← CopierTuiles – la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser()   x 2   26 o sur place, dans va_a() + AdresseCase (10 o)
effacer() x 1   15 o sur place, dans va_a() + AdresseCase (10 o), EffacerCases (12 o)
image()   x 2    6 o sur place, dans va_a(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
va_a()   x 4   89 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <va_a>      // mène une tuile jusqu'à une case
#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A
```

2. UNE VARIABLE — X

```
uint8_t x = 0;      // la colonne du A
```

3. UNE VARIABLE — Y

```
uint8_t y = 0;      // sa ligne
```

4. UNE VARIABLE — XB

```
uint8_t xb = 0;     // la colonne du B
```

5. UNE VARIABLE — YB

```
uint8_t yb = 10;    // sa ligne
```

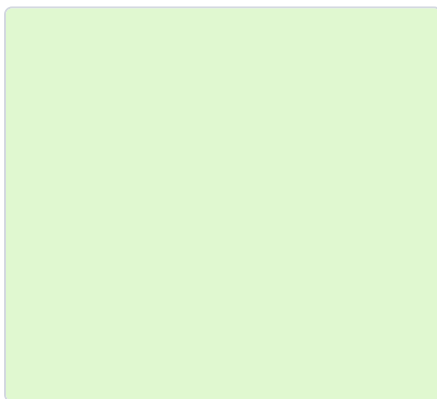
6. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    va_a(x, y, ALPHABET[0], 10, 5);
    va_a(x, y, ALPHABET[0], 0, 0);
    va_a(xb, yb, ALPHABET[1], 10, 15);
    va_a(xb, yb, ALPHABET[1], 0, 10);

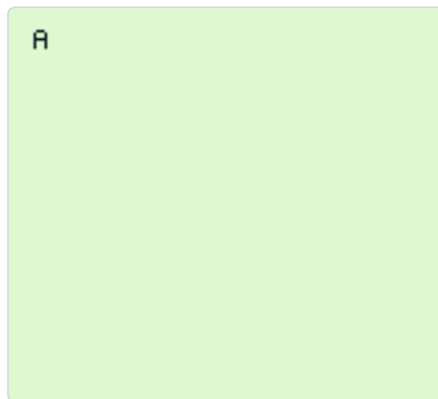
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

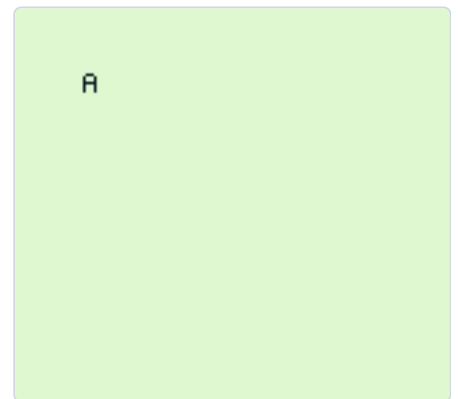
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



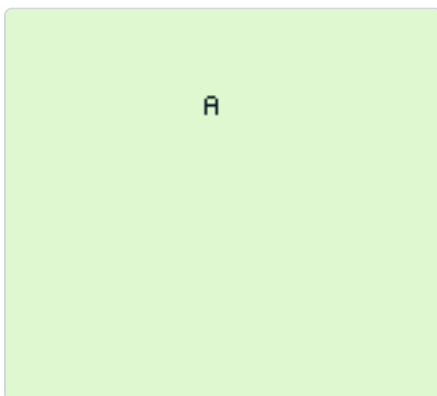
la toute première image — la console vient de commencer — image 1



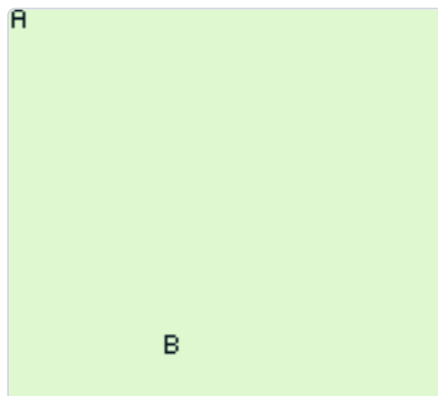
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Le A va et revient en haut, puis le B en bas.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A finit en (0, 0) (0,0)

✓ le B passe par (10, 15)

✓ le B finit en (0, 10) (0,10)

Ensuite : **0.29.3. La fonction `chaque()` — toutes les `n` millisecondes** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++